ANDROID MEETUP · 9 JULY 26

TestBalloon Attempt to Lift Your Test Infra

One Kotlin DSL for unit, Robolectric, instrumented and snapshot tests — born at KotlinConf 2025.

Eugen Martynov · github.com/infix-de/testBalloon · Apache-2.0


КИНОЛОГ
ПОШУК ЗАГИБЛИХ
Дозор
НА ШОЛОМІ
200

ABOUT ME

Eugen Martynov

Android developer who has been thinking about testing longer than most people care to admit.

2012-2014 — First Android testing and first Robolectric talks.

JUnit 3 → 5 — Every JUnit generation, plus Hamcrest, Truth, AssertJ and frameworks that didn't survive. Followed Spek closely.

Blog series — Deep-dive series on test fixtures for Android modules — scope, sharing, issues and working setup.

Snapshots — Recent post: practical snapshot testing rethinking for Android Paparazzi setup — faster, less flaky, more useful.

Today — Migrating NowAndroid to TestBalloon — and presenting what I found at this meetup.



AGENDA

What we'll cover tonight

01

The Landscape

Why Android testing still feels like three jobs in one

02

Meet TestBalloon

What it is, where it came from, how it thinks — and the core concepts

03

Live Demo

Migrate a real JUnit 4 test to TestBalloon — live on stage

04

TestBalloon on Android

Compose, Robolectric, instrumented, performance, reports, IDE plugin, NowAndroid

CHAPTER ONE

The Landscape

Why testing on Android still feels like three jobs in one.

THE LANDSCAPE

The Android testing matrix

Unit Tests	**JUnit 4** most common	**JUnit 5** growing	**kotlin.test** KMP	**Kotest** KMP
Integration / Instrumented	**Instrumented** JUnit 4 runner	**Instrumented** JUnit 5 (ext)	**Robolectric** JUnit 4 / 5	
Snapshot Tests	**Paparazzi** JUnit 4	**Roborazzi** JUnit 4 / JUnit 5 *		

** Roborazzi ships a separate roborazzi-junit5 artifact — JUnit5 support exists but is not the default path.*

The pain points

Java-first foundation

JUnit 4 & 5 were designed for Java. Kotlin uses them via adaptation — not native design. Annotations, not lambdas; classes, not functions.

Coroutines need a wrapper

Every suspending test body requires runTest { } plus manual dispatcher setup. There is no native suspend support in the standard runners.

Parameterized and Nested test complexity

@ParameterizedTest, @MethodSource, @CsvSource, ArgumentsProvider… a whole vocabulary just to run the same test with different inputs.

Hardish to extend in JUnit 4

@Rule and @ClassRule chains compose poorly. Adding cross-cutting behaviour (timing, retries, custom lifecycle) means wrestling with rule ordering.

JUnit 5 on Android: unofficial

Instrumented tests still depend on the JUnit 4 runner. JUnit 5 support is community-maintained, not official — and it shows in CI edge cases.

Snapshot libs locked to JUnit

Paparazzi, Roborazzi etc. are JUnit rule or extension wrappers. Switching frameworks means re-integrating each tool from scratch.

CHAPTER TWO

Meet TestBalloon

A single, Kotlin-native DSL for every one of those lanes.

MEET TESTBALLOON

What exactly is it?

- A Kotlin-first test framework — not a JUnit extension, a full DSL.
- Powered by a Kotlin compiler plugin, so suites read like plain Kotlin code.
- Apache-2.0 licensed, maintained by infix Software-Systeme GmbH.
- Runs alongside your existing JUnit-based tests — no big-bang rewrite.
- Ships with a dedicated IntelliJ IDEA plugin (JetBrains Marketplace).

1.0.1

Latest stable release — semver guaranteed

Reached a stable 1.0.0 with semantic versioning — the API you migrate to today is the API you keep.

Quick start: your first suite

AccountTests.kt

```
val AccountTests = testSuite {

    test("deposit increases the balance") {
        val account = Account(balance = 100)
        account.deposit(50)
        assertEquals(150, account.balance)
    }

    test("rejects a negative amount") {
        val account = Account(balance = 100)
        assertThrows { account.deposit(-10) }
    }
}
```

- **Blank line before each test**

  Gives the eye a natural pause — as readable as seeing @Test on its own line.

- **Body always on new lines**

  Never inline. test("…") { assertion } is the JUnit anti-pattern equivalent.

- **Flat over deep nesting**

  Prefer separate top-level suite properties over nested testSuite { testSuite { } } chains.

- **Names are full sentences**

  No backtick length limits — write the whole intent: "rejects a negative amount".

Suites nest naturally

Nested suites

```
val AccountTests = testSuite {
    testSuite("deposits") {
        test("increases balance") { ... }
        test("rejects negatives") { ... }
    }
    testSuite("withdrawals") {
        test("decreases balance") { ... }
    }
}
```

- Suites are plain Kotlin properties — multi-level nesting is just nested function calls.
- Execution order is source order by default — no @Order annotation required.
- Dynamic/data-driven tests fall out naturally: generate test() calls in a loop.

Fixtures: setup & teardown, typed

Fixture with auto-close

```
val AccountTests = testSuite {
    val account by testFixture { Account(balance = 100) }
        .closeWith { it.shutdown() }

    test("deposit increases balance") {
        account().deposit(50)
        assertEquals(150, account().balance)
    }
}
```

Fresh value per test, or shared per suite

asParameterForEach

```
val repo by testFixture {
    StarRepository()
}.asParameterForEach()

test("a") { repo().star(1) }
test("b") { repo().star(2) }
// each test: a NEW repo
```

asParameterForAll

```
val repo by testFixture {
    StarRepository()
}.asParameterForAll()

test("a") { repo().star(1) }
test("b") { repo().star(2) }
// shared across the suite
```

Same fixture declaration, one suffix changes the lifecycle — no separate API to learn.

Coroutines, not bolted on — built in

- Test bodies are suspend functions by default — no runTest {} wrapper to remember.
- Each TestSession runs on kotlinx.coroutines.test.TestScope unless you override it.
- TestConfig.coroutineContext() lets a suite or test swap dispatcher or context.
- TestConfig.invocation(Concurrent) opts a suite into structured concurrent execution.

Suspend by default

```
test("loads from network") {
    val data = api.fetchUser(id)
    assertEquals("Ada", data.name)
}

// no runTest { }, no
// TestScope wiring needed
```

Parameterization is just Kotlin

Data-driven tests

```
val sizes = listOf(0, 1, 10, 1_000)

val BufferTests = testSuite {
    sizes.forEach { size ->
        test("allocates buffer of size $size") {
            assertEquals(size, Buffer(size).capacity)
        }
    }
}
```

No @ParameterizedTest, no @MethodSource, no argument-provider classes — just iterate and call test().

LIVE DEMO

Let's migrate a test together

Take one real JUnit 4 test class from a module we know — port it to TestBalloon, run it, check the report.

CHAPTER FOUR

TestBalloon on Android

Robolectric, instrumented tests, performance, reports, IDE plugin, and a real migration.

4

How it runs on Android, specifically

- Hooks into Android's existing JUnit 4 test runner — for both host-side unit tests and device-side instrumented tests.
- Runs alongside other JUnit-based tests already in the same module — no big-bang rewrite required.
- Execution is sequential only on Android, due to instrumentation constraints — not a TestBalloon limitation.
- JUnit4 @Rule-based tooling keeps working via JUnit4RulesContext, including Compose UI testing rules.
- Environment variables for device tests are simulated through instrumentation arguments.

Compose UI testing — JUnit4RulesContext

Compose test suite

```
val ComposeTests by testSuite {
    testFixture {
        object : JUnit4RulesContext() {
            val compose = rule(createComposeRule())
        }
    } asContextForEach {
        test("button tap shows success") {
            compose().setContent { MyButton() }
            compose().onNodeWithText("Tap me")
                .performClick()
            compose().onNodeWithText("Done!")
                .assertExists()
        }
    }
}
```

- JUnit4RulesContext is a TestBalloon bridge for any @Rule-based tool — rule() handles setUp and tearDown automatically.
- asContextForEach gives each test a fresh ComposeTestRule instance, matching @get:Rule semantics.
- Same pattern works for MockWebServer, Robolectric ActivityScenario rules, or any other JUnit4 rule.

Robolectric, configured declaratively

Robolectric suite

```
class MyRobolectricTests : RobolectricTestSuiteContent({
    test("activity renders title") {
        val activity = Robolectric.buildActivity(MainActivity::class.java)
        assertEquals("Now in Android", activity.get().title)
    }
})

robolectricTestSuite<MyRobolectricTests>("main activity",
    testConfig = TestConfig.robolectric { sdk = 34; qualifiers = "en-rUS" })
```

Reports, coverage & CI — standard tooling, not bespoke

- Reports: standard Gradle JUnit XML + HTML — same as any JUnit test. One caveat: all tests appear under a single TestBalloonJUnit4 entry at the module level, requiring one extra click to reach individual classes.
- Filtering: --tests "com.example.Suite|inner suite|*" or the TESTBALLOON_INCLUDE_PATTERNS instrumentation argument for device tests.
- Coverage: JaCoCo .exec files are written automatically during testDemoDebug — no extra configuration. Apparent coverage differences vs. a JUnit baseline are usually explained by Kotlin or JaCoCo version bumps, not actual coverage changes.

An IDE plugin, not just a CLI tool

JETBRAINS MARKETPLACE

IntelliJ IDEA plugin

`Plugin ID 27749 — search “testballoon”`

Installs alongside the Kotlin plugin — no separate runner configuration needed.



Gutter run icons

Click to run or debug any test() or testSuite() right from the editor margin.



Dedicated result tree

Browse pass/fail status per suite, not just a flat method list.



Jump to source

Navigate straight from a failure to the failing assertion.



Stack trace folding

Collapses framework noise so failures are easy to read.

Known limitation: “Rerun Failed Tests” is not yet supported — re-run the suite instead.

Migrating "Now in Android" — what worked, what's stuck

WHAT MIGRATED CLEANLY

- JVM unit tests: testSuite {} replaces class; testFixture replaces @Before/@After.
- Robolectric suites via RobolectricTestSuiteContent — sdk and qualifiers inline.
- Gradle report: same HTML/XML as before, coverage unaffected.

STUCK POINTS

- Hilt — HiltTestApplication is incompatible with TestBalloon's runner; Hilt hooks into the Application class in a way TestBalloon's runner doesn't support yet.
- Snapshots — Paparazzi/Roborazzi are JUnit rule wrappers; rule() partially bridges, but the capture lifecycle doesn't fully map. Leave snapshot tests on JUnit 4 for now.

NowAndroid: TestBalloon vs. JUnit baseline

JUnit 4 (main)

23.9s

avg · 5 measured runs

`range  22.2s — 25.3s`

NowAndroid · testDemoDebug · 143 tests

TestBalloon 1.0.1

21.8s

avg · 5 measured runs

`range  18.7s — 25.6s`

NowAndroid · testDemoDebug · ~189 tests (same logic)

~9% faster on average · ranges overlap · not statistically conclusive at 5 iterations · Gradle Profiler 0.24.0 · 2 warm-ups + 5 measured · --no-build-cache

Compartments keep parallelism sane

Concurrent | **Default** | **RealTime** | **Sequential** | **MainDispatcher**

- TestCompartment groups top-level suites so unrelated suites don't fight over isolation or threads.
- TestSession is the module-wide root config — sequential, Dispatchers.Default, TestScope by default.
- Warning straight from the docs: never combine TestBalloon's own concurrency with Gradle's maxParallelForks.

Is it worth it?

- One DSL for unit, Robolectric, instrumented and Compose — three jobs become one.
- Coroutines first-class: no runTest{} wrapper, TestScope in every test by default.
- JUnit4 rules keep working via JUnit4RulesContext — no big-bang rewrite needed.
- IDE plugin: gutter icons, result tree, jump-to-failure — proper tooling support.
- Stable API (1.0.x semver), Apache-2.0, actively maintained.

KNOWN GAPS

Hilt

HiltTestApplication is incompatible with TestBalloon's runner.
Workaround: keep Hilt tests on JUnit 4 — they coexist fine.

No big resources behing

Small company and a bunch of adopters.

Android experience

Android JUnit4 situation and lack of Android expertise and motivation.

Resources to keep going

GitHub

`github.com/infix-de/testBalloon`

Docs

`infix-de.github.io/testBalloon`

Migration branch

`github.com/emartynov/nowinandroid/tree/feature/convert-to-testballoon-skill`

IntelliJ plugin

`Marketplace ID 27749 · "testballoon"`

QUESTIONS & COMMENTS

Let's talk TestBalloon.

github.com/infix-de/testBalloon • Docs: infix-de.github.io/testBalloon